

GRAPH COLORING IN GRAPHBLAS

An Undergraduate Research Scholars Thesis

by

HEMANTH MUKESH

Submitted to the Undergraduate Research office at
Texas A&M University
in partial fulfillment of requirements for the designation as an

UNDERGRADUATE RESEARCH SCHOLAR

Approved by
Faculty Research Advisor:

Dr. Timothy Davis

May 2025

Major:

Computer Science

Copyright © 2025. Hemanth Mukesh.

RESEARCH COMPLIANCE CERTIFICATION

Research activities involving the use of human subjects, vertebrate animals, and/or biohazards must be reviewed and approved by the appropriate Texas A&M University regulatory research committee (i.e., IRB, IACUC, IBC) before the activity can commence. This requirement applies to activities conducted at Texas A&M and to activities conducted at non-Texas A&M facilities or institutions. In both cases, students are responsible for working with the relevant Texas A&M research compliance program to ensure and document that all Texas A&M compliance obligations are met before the study begins.

I, Hemanth Mukesh, certify that all research compliance requirements related to this Undergraduate Research Scholars thesis have been addressed with my Faculty Research Advisor prior to the collection of any data used in this final thesis submission.

This project did not require approval from the Texas A&M University Research Compliance & Biosafety office.

TABLE OF CONTENTS

	Page
ABSTRACT.....	1
ACKNOWLEDGEMENTS.....	3
NOMENCLATURE	4
1. INTRODUCTION	5
1.1 Graph Coloring	5
1.2 GraphBLAS.....	6
1.3 Motivation.....	8
2. RELATED WORK.....	10
3. METHODS	11
3.1 Specifications.....	11
3.2 File Structure	11
3.3 Graph File Formats	12
3.4 Greedy Algorithm – NetworkX Baseline	13
3.5 Independent Set Algorithm – GraphBLAS.....	14
3.6 Maximal Independent Set Algorithm – GraphBLAS.....	17
3.7 Graphs.....	19
4. RESULTS.....	20
4.1 Algorithm Runtime	20
4.2 Number of Colors	22
5. CONCLUSION.....	24
5.1 Challenges and Limitations	24
5.2 Future Work.....	25
REFERENCES	27
APPENDIX: ALGORITHMS	28

ABSTRACT

Graph Coloring in GraphBLAS

Hemanth Mukesh
Department of Computer Science and Engineering
Texas A&M University

Faculty Research Advisor: Dr. Timothy Davis
Department of Computer Science and Engineering
Texas A&M University

In computer science, many traditional approaches to graph algorithms rely on iterative and sequential techniques applied to adjacency lists. However, with the emergence of GraphBLAS, a linear algebra-based framework for graph algorithms, a new, more parallel approach to solving graph problems can be explored. GraphBLAS provides a powerful alternative through a robust framework that defines a set of matrix and vector operations to perform operations on graphs. Using concepts from linear algebra, GraphBLAS unlocks the potential for increased performance on several algorithms suited for parallel implementations, including graph coloring.

Graph coloring is a fundamental problem in graph theory with applications primarily related to scheduling, such as register allocation or resource management. The concept of graph coloring is to color each node or edge of a graph such that no two adjacent nodes share the same color. While a successful coloring can be achieved with coloring each node a different color, coloring algorithms aim to minimize the number of colors as well as the runtime.

This work explores the GraphBLAS implementation of two graph coloring algorithms, including the independent set algorithm and the maximal independent set algorithm, as well as the NetworkX greedy implementation. Next, we test and compare the performance of these algorithms against each other, looking at the algorithm runtime and the number of colors used. We find that the maximal independent set algorithm in GraphBLAS proves to be much more time efficient on large graphs compared to the NetworkX greedy algorithm.

ACKNOWLEDGEMENTS

Contributors

I would like to thank my faculty advisor, Dr. Davis, for his guidance and support throughout the course of this research. I also thank the other researchers under Dr. Davis for their support throughout the research process.

The graph matrix data used in this research as well as the maximal independent set algorithm was provided by Dr. Timothy Davis. All other work conducted for the thesis was completed by the student independently.

Funding Sources

No funding was received.

NOMENCLATURE

BLAS	Basic Linear Algebra Subprograms
NP	Nondeterministic Polynomial Time
API	Application Programming Interface
MIS	Maximal Independent Set
CUDA	Compute Unified Device Architecture

1. INTRODUCTION

1.1 Graph Coloring

Graph coloring is a widely studied problem in graph theory with a wide range of real-world applications. It involves assigning colors to elements of a graph such that no two adjacent elements share the same color. The most common element used is the vertices of a graph, although the edges can also be colored depending on the application. Other variations can also change the depth of the color constraint – such as specifying that no elements up to 2 hops away share the same color, versus just the neighbors (that are only 1 hop away). These problems have several applications in representing real-world situations such as resource allocation, scheduling tasks, frequency assignments in networks, and even in solving sudoku puzzles. To expand on an example, compiler optimization is a specific computing application where graph coloring algorithms are used, specifically in register allocation. In this scenario, program variables are treated as vertices, and edges are placed if any pair of variables are live at the same time. By coloring this graph with the fewest number of registers, a compiler can efficiently allocate registers without any conflicts between variables. Similarly, in wireless networking, frequency assignment problems can be modeled as coloring problems. There are only so many frequencies that can be used, so frequency assignment policies need to be efficient. By representing network cells or routers as nodes, and drawing edges between them, we can calculate a set of frequencies such that no two network nodes will conflict with each other, while keeping the total set of frequencies to a minimum. This wide applicability across disciplines highlights the importance of optimizing the time and quality of coloring algorithms.

The coloring problem we will be focusing on is the traditional, vertex based 1 hop coloring problem. The goal is to minimize the number of colors required to fulfill the requirements of a valid coloring and minimize algorithm runtime. Currently, finding a true minimum coloring is an NP-Complete problem, meaning that there are no known solutions to solve this problem in a reasonable amount of time. Algorithms that currently tackle this problem give approximate solutions, meaning that while they finish within a reasonable amount of time, they do not always find the best solution possible. In this paper, we will be going over the different approaches to approximate this solution, and how some can be parallelized with GraphBLAS.

1.2 GraphBLAS

GraphBLAS is an API specification that defines matrix and vector operations from linear algebra, which can be applied to adjacency or incidence matrices to perform graph algorithms [1]. GraphBLAS is built around semirings, which define algebraic behaviors that can be applied to matrices and vectors. The benefit of representing graphs as matrices versus nodes is that GraphBLAS can execute operations on matrices in parallel, while traditional graph algorithms often work on each node sequentially. However, not all algorithms can be parallelized. Depth-first search, for example, is sequential by nature and will not see a significant speed up with a parallel approach. On the other hand, breadth-first search can be sped up through parallel approaches in GraphBLAS. Traditionally, breadth first search algorithms execute operations on each node sequentially. However, due to the nature of the algorithm, some operations can be executed at the same time, reducing the time to complete this algorithm. GraphBLAS can also reduce computation time when working with sparse graphs, which are graphs that have significantly less edges than the maximum possible number of edges. In GraphBLAS, these

graphs are stored in formats that optimize the space used and include pre-computed data to accelerate operations.

One feature of GraphBLAS that sets it apart is its ability to abstract away the low-level complexity often associated with parallel computation. Parallel algorithms often require complex and highly specialized code, including thread synchronization, data partitioning, and memory consistency. This can be difficult to write, and the complexities with this approach can introduce various errors that need to be accounted for and scaling issues. However, through GraphBLAS, many of these issues are resolved through optimized backend implementations and kernels that automatically adapt to data sparsity and structure, all so that the user does not need to write that kind of code themselves and can focus on the logic of their algorithms. Furthermore, by representing graphs as matrices, parallel algorithms become easier to understand from a mathematical viewpoint. Algorithms built on top of GraphBLAS can be relatively simpler compared to other parallel approaches yet just as fast, if not faster.

1.2.1 Limitations

As mentioned earlier, not every graph algorithm can be easily adapted into the models that GraphBLAS provides. Algorithms that depend on decisions per node or irregular flows have difficulty converting to GraphBLAS. Expanding on the example earlier, depth-first search is an algorithm that is sequential by nature. Finding the next node to explore depends on the result of the previous depth-first search subroutine, and every node following that is similar. These per node decisions are characteristics of an algorithm that will not be able to benefit from a GraphBLAS approach. On the other hand, breath-first search explores all of the neighbors of its initial set in one step, as opposed to depth-first search's single node in one step. This fact can be used to rewrite the breath-first search algorithm as a series of matrix operations that tell

GraphBLAS to operate on all neighbors simultaneously. This approach successfully parallelizes the algorithm, and GraphBLAS is able to provide benefits to algorithm runtime. While GraphBLAS can greatly accelerate graph algorithms, using it effectively requires a thorough mathematical reformulation of the problem.

1.2.2 SuiteSparse

SuiteSparse:GraphBLAS (referred to as SuiteSparse in this paper) is a C implementation of the GraphBLAS standard as of writing that is fully compliant with the GraphBLAS standard and is the official reference implementation of GraphBLAS [2], [3]. It was developed by Dr. Timothy Davis at Texas A&M University. SuiteSparse was the implementation used in this work, although this paper will use the term “GraphBLAS” when discussing the implementation at a high level.

1.2.3 LAGraph

LAGraph is a library and test harness for graph algorithms developed in SuiteSparse [4]. The motivation behind LAGraph is to provide a way for developers to use the power of GraphBLAS without writing the graph algorithms themselves. LAGraph is designed to be user-friendly, and its official algorithms are well documented. These official algorithms are available in the library and are ready for use after the download and installation of LAGraph. Note that this requires SuiteSparse to be installed as well. The algorithm in this paper will be available in the unofficial experimental folder of LAGraph, alongside other experimental algorithms developed by fellow researchers.

1.3 Motivation

Optimized algorithms are becoming increasingly relevant in today’s world. As the demand for more data increases, all aspects of technology need to scale to process such data.

Graph coloring is a problem widely recognized in technology, and the need for practical solutions for large graphs will continue to increase as long as our data consumption continues to increase. The motivation behind this research is to develop coloring algorithms that capitalize on the improved computational time that GraphBLAS offers on large graphs. After evaluating the performance of the algorithm in terms of time as well as quality, the coloring algorithm could then be added to the growing LAGraph repository, where developers and users of LAGraph can utilize or build upon this algorithm. This research has the potential to highlight any weaknesses or constraints in GraphBLAS, which could benefit the development team by providing an example for an area that could be improved, helping make GraphBLAS more robust. GraphBLAS research also has the potential to showcase incredible results, creating more interest in the software and encouraging more development with it.

2. RELATED WORK

A group of researchers published a paper titled “Graph Coloring on the GPU”, which implements and analyzes the performance of coloring algorithms built in GraphBLAS and Gunrock [5]. Gunrock is “a CUDA library for graph-processing specifically designed for the GPU” [6]. This technology is out of the scope of this paper, but more information about Gunrock can be found on their official website. The paper presents multiple GraphBLAS algorithms for graph coloring, starting with an independent set approach, then building a maximal independent set algorithm, then finally the Jones-Plassman algorithm. The pseudocode for the independent set algorithm in “Graph Coloring on the GPU” was closely replicated as a starting point for this research. However, there were a few operations in their approach that were inefficient and unnecessary. The independent set algorithm in this current paper has been built with fixes to the problems in the original algorithm. However, we will not be presenting a comparison of the results between these two algorithms.

The independent set algorithm in “Graph Coloring on the GPU” was loosely defined solely through pseudocode, and there were no official downloads for the exact code. Therefore, any implementation of their pseudocode cannot be confirmed to be an exact copy of that algorithm. Any results from that algorithm cannot be attributed to that paper, so they will not be discussed in this paper. We will instead discuss the code in the improved version and explain where our improvements are.

3. METHODS

3.1 Specifications

The algorithms in this paper were developed on my personal computer running Windows 11. The editor primarily used was VSCode, and a public Github repository acted as cloud storage for the code. A link to the Github repository is included in the references [7]. The version of SuiteSparse:GraphBLAS used was version 9.4.3, released December 20th, 2024. A developing LAGraph version was used – v1.2. This branch received constant updates, which added new functionality and fixed various bugs. More information on the commit histories and versions of LAGraph used can be found in the Github repository history and forked repository [4]. Testing was done on backslash – a server at Texas A&M University owned by Dr. Tim Davis. Backslash is a powerful compute node with a 12-core Intel Xeon E5-2695 v2 CPU, containing 12 cores clocked at 2.40GHz, one socket, 24 total threads, and 768 GB of RAM. While the server was not reserved exclusively for this research, all algorithms were executed without any background processes using significant amounts of resources that could have hindered the results.

3.2 File Structure

Each LAGraph algorithm involves an algorithm file, a test file, and a benchmark file. This modular approach to development helps with understandability and maintainability. Furthermore, LAGraph will automatically detect the test and benchmark files based on their folder location and naming convention. It will also compile and execute them along with the other tests and benchmarks when running overall tests and benchmarks across all of LAGraph. The algorithm file includes the main code of the algorithm, written primarily with GraphBLAS calls. The algorithms take an LAGraph matrix as an input and output a vector of colors as well as

the total number of colors used. These 2 metrics are measured in the results section. The test file is primarily for code coverage and ensuring the algorithm works as intended. In the test code, the algorithm was executed on various matrices. The results were then verified by a C function that checked every node's color was different than its neighbor's color. This portion was written in raw C, with GraphBLAS only used to unpack the matrix. This decision was made because testing GraphBLAS with itself is not the best idea. The tests were written in raw C to ensure that even if GraphBLAS had any errors or bugs, the tests would still be safe and run as intended. The benchmark file is for running the algorithm on large and extremely large graphs, primarily for recording the time spent as well as the number of colors used. Data files containing matrices of graphs were used during the testing and benchmarking of these algorithms. While many of these data files were already available within LAGraph, smaller data files with simpler graphs were developed for bug testing and edge case testing. Finally, header files were also edited to include the addition of the new files and methods.

There is one algorithm that did not use GraphBLAS – the NetworkX greedy approach. This algorithm serves as the baseline for the rest of this research. It uses a sequential, greedy approach to graph coloring, and is available in a Python library called NetworkX [8]. This is the only algorithm that was coded in Python.

3.3 Graph File Formats

There are two common data structures to represent a graph: an adjacency list or an adjacency matrix. In an adjacency list, each node in the graph stores a list of its neighbors. An adjacency matrix uses a matrix where nodes represent both the rows and the columns. A value of 0 represents no edge between the nodes at the specific row and column, and a 1 represents an edge between those nodes. The advantage of adjacency matrices is that they are easier to

understand and visualize linear algebraic operations on. However, the extra space for storing 0s is not efficient in a computer, especially for graphs that have millions of nodes and connections. There are two file formats that will be used in this paper to store graphs – Matrix Market and a custom LAGraph Binary format. The Matrix Market format provides “a simple mechanism to facilitate the exchange of matrix data” and is based on ASCII [9]. The extension is ‘.mtx’, and every matrix that has been tested was initially in a ‘.mtx’ format. However, due to the slow loading of matrices in ‘.mtx’ format, LAGraph has its own file format that is in binary, as opposed to ASCII, with the file extension of ‘.grb’. Using ‘.grb’ files dramatically speeds up the time to read a graph and load the contents into an LAGraph object. Because this file format is specific to LAGraph, it was not available in the NetworkX greedy algorithm.

3.4 Greedy Algorithm – NetworkX Baseline

The greedy algorithm for graph coloring is a sequential algorithm and would not benefit from a GraphBLAS implementation. Instead, it will be used as the baseline for comparing the other GraphBLAS algorithms. This greedy coloring algorithm is from NetworkX, a popular Python graph library. There are a variety of strategies that this specific greedy coloring algorithm allows, which affect the order in which the nodes are colored. However, for the purposes of this research, the default strategy was used and did not specify any specific ordering. Algorithm 1 (see Appendix) outlines the pseudocode for this approach.

The input is a graph in a file format, and the output is an array of colors that correspond to a valid coloring. While the code behind the algorithm is relatively simple, there are no parallel techniques used here.

3.4.1 *Implementation Details*

Firstly, the algorithm reads the Matrix Market file to get the contents of the graph. It is then transferred into a NetworkX graph object, and then the algorithm is run. The exact code for the method can be found in the NetworkX documentation [8]. For a brief overview, the nodes are first ordered according to the strategy used. Then, the algorithm sequentially evaluates what color to assign to each node by examining the nodes neighbors and assigning the color as the lowest number that is not already used with the neighbors. The return value is a Python list of numbers that correspond to a color for each node. The runtime of the algorithm is measured and stored. Furthermore, the number of unique colors is calculated from the returned list. Those two metrics are stored and analyzed later in the results section.

3.5 **Independent Set Algorithm – GraphBLAS**

The Independent Set algorithm was the first algorithm written in GraphBLAS in this project and is the base for the remaining algorithms. The following maximal independent set algorithm and the Jones-Plassman algorithm add subroutines to the independent set algorithm that help achieve greater efficiency in terms of speed and colors used.

The algorithm colors nodes by attempting to find an independent set. An independent set is a set of nodes that are not connected to each other. To find such a set, the algorithm assigns random weights to each node and then compares the weights between each pair of nodes to find the nodes that have a higher weight than all their neighbors, or in other words, the nodes that have “won” over their neighbors. In this way, 2 nodes connected by an edge cannot both be in the set – one must lose to the other. After finding an independent set, the algorithm colors those nodes and then repeats the process until all nodes are colored.

The reason behind using random weights is to break any symmetry in the graph. Without randomness, the algorithm could behave poorly because it wouldn't be able to choose which nodes gets to be in the set. However, this also introduces non-determinism into the algorithm, meaning that two runs on the same graph might yield different colorings. To combat that, an integer input is used as the seed for the pseudo-random algorithm. Inputting the same seed will result in the same pseudo-randomness every time.

3.5.1 *Implementation Details*

Algorithm 2 (see Appendix) details the GraphBLAS operations to execute the algorithm. The algorithm starts with an input of an `LAGraph_Graph` object. This is an `LAGraph` object that holds the graph contents and other information. In our benchmarking, the algorithm was fed an `LAGraph` object that contained graph information from a '.grb' file. These files were much faster to read in as opposed to Matrix Market files. The details of reading in the files and putting them into a `LAGraph` object are not in the pseudocode above but can be viewed in the Github code repository [7].

The first 2 lines of the algorithm are statements to initialize the 3 vectors we will be using: `max_weights`, `in_curr_subset`, and `weight`. We do this to set the type and size of these vectors. All vectors are of size `n` and `max_weights` and `weight` are both numerical types. The vector `in_curr_subset` is of type `bool`. The `weight` vector is initialized with random numbers using a random function from `LAGraph`.

After initialization, the `mxv`, or matrix times vector, occurs next. Normally in linear algebra, this is called a dot product and involves the multiplication of a set of elements from the graph and the vector, and then the addition of all those elements into one value for part of the final vector. In GraphBLAS, we can use different operations instead of multiplication and

addition by specifying a semiring. A semiring defines a multiplicative and an additive operator. In this function, the matrix is the adjacency matrix of the LAGraph object, $G \rightarrow A$, and the vector is the weights of the nodes, `weight`. The max second semiring is used in this `mxv`. The semirings follow an “add_mult” format, so for this semiring, the multiplicative operator is “second” and the additive operator is “max”. The multiplicative operator second means that if there is a value in the adjacency matrix, then we simply choose the corresponding weight value. Here, the operator is selecting the weights of all of the node’s neighbors. Afterwards, the additive operator is max, so we take the max weight of the node’s neighbor. The resulting vector is the biggest neighbor’s weight for each node.

In the next operation, the algorithm calls an `eWiseMult`, which is an element wise multiplication. The “multiplication” part of the operation can be a little misleading because no multiplication happens. This operation is an element wise operation between two vectors but can be performed in two ways. When one vector does not have a value, we can either include that in the final result, leading to a result similar to addition, or we can exclude that in the final result, leading to a result similar to multiplying by 0. In this algorithm, we might see no entries for the original weight vector as we color nodes in and exclude them from further calculations. We do not want to include that in the final result, so we use the “mult” version. The operator used is greater than, which is abbreviated to GT. This operation calculates whether the node weight in the vector weights is bigger than its biggest neighbor in `max_weights`. The result of this operation is a Boolean matrix that we call `in_curr_subset`. This is the independent set that we will use to color.

The `GrB_select` in line 5 is to clean up the `in_curr_subset` vector. The vector is currently dense, with either a 0 or a 1 in each spot. The `GrB_select` will select all items with a certain

value, true/1 in this instance, and keep those items in the vector while deleting the rest. This sparsifies the vector, which speeds up the next step by using a structural mask rather than a valued mask. On line 6, we do a check to make sure that the `in_curr_subset` vector contains values. If it does not then we assume that we are done with the coloring and can quit the loop.

Next, on line 8, we assign the color. We use a `GrB_assign` to assign the current color onto the `local_color` vector that we will eventually return. However, we don't want to color every node, just the nodes in the independent set. Therefore, we use a mask of the independent set (`in_curr_subset`). This mask will be structural because we have done a `GrB_select` on only the true values, so if there is a value, then it must be true.

Finally, we use a `GrB_assign` to set the weight of the colored nodes to 0. This causes the weight calculations on that node to cancel out to 0, making sure that those nodes are not considered again.

3.6 Maximal Independent Set Algorithm – GraphBLAS

The maximal independent set algorithm is very similar to the independent set algorithm. To make the set maximal, we simply run the independent set algorithm multiple times until no new candidates can be found. The sets from the multiple iterations of the independent set algorithm are combined to create a maximal independent set.

3.6.1 Implementation Details

The pseudocode is available in Algorithm 3 (see Appendix), while the code for the maximal independent set function can be found in the LAGraph Github [4]. Taking a deeper look at the maximal independent set function, the main differences between this one and the independent set algorithm are the loop structures and the extra setup at the end. Because we are doing multiple independent sets, we need to loop the independent set algorithm multiple times.

Furthermore, we can remove immediate neighbors of the last independent set from candidates, since a neighbor of an independent set cannot also be part of the same set. Another interesting situation that we must account for is the situation that multiple nodes have the same random value as their weight. This could cause a stall – the code expects one node to be added to the independent set, but neither ‘wins’ against the other. In this situation, the code will regenerate random numbers before trying again. If the nodes continue to equal the same value 32 times in a row, then the program quits, as something must be terribly wrong.

With these additions, the maximal independent set algorithm can find bigger sets at a time compared to the independent set algorithm, meaning that the number of colors must be lower, if not the same. While this does cost extra time per iteration, there are lower iterations overall, so time will be saved with this approach.

Lastly, this maximal independent set algorithm incorporates push/pull optimization when doing matrix and vector multiplication. This technique is used when the sparsity pattern of the objects change over time. When the algorithm starts, A is a sparse matrix and weight is a dense vector. During the early iterations, the mxv multiplication is more efficient with a sparse matrix and a dense vector. However, as the algorithm progresses, weight will become increasingly sparser as entries are marked as colored and then deleted from weight. Switching to a vxm multiplication will speed up the process and reduce unnecessary computation. This dynamic switching mechanism can provide significant runtime improvements for situations where sparsity changes. However, GraphBLAS does not automatically switch between the two. In order to account for this manually, the algorithm will check the sparsity of the weight vector prior to any multiplication calls. Based on that information, it will choose either the mxv multiplication or the vxm multiplication.

3.7 Graphs

To simply test for correctness and code coverage, the test files used smaller matrices that were easier to track by hand and run quickly. However, the benchmarking files used much larger matrices and ran the algorithms on the backslash server. There were 4 graphs used for benchmarking: as-Skitter, com-Youtube, com-LiveJournal, and GAP-road. The first matrix, as-Skitter, is an internet topology graph made up of traceroutes that were ran daily from 2005. Com-Youtube is a graph of a connected community of users on the Youtube platform who are friends with each other. Com-LiveJournal is the same concept, but for a different platform called LiveJournal. Finally, GAP-road is a matrix of all the roads in the USA, where nodes represent their connections, and the edge weights represent distances. While all these graphs have different structures, they are all undirected, symmetric graphs. This is a requirement for coloring algorithms – it doesn't make sense to color a directed graph. Another property to consider is self-edges. These are edges that connect a node to itself and are not relevant to coloring either. For the GraphBLAS algorithms, the algorithms require that the user deletes self-edges and creates a symmetric graph prior to passing in the graph to the function. In the future, if these algorithms are approved to become official within LAGraph, a more user-friendly algorithm will be developed to automatically remove self-edges and ensure symmetry in the graph.

The structures of these graphs do vary, and these differences can affect the results. The extreme case is GAP-road, which is a sparse graph due to the structure of road networks in the US. It is rare for more than 2 roads to intersect with each other, leading to most nodes having very few edges. This high level of sparsity means that finding independent sets become relatively easy, which has an impact on the results that will be explored in the next section.

4. RESULTS

Each algorithm will be evaluated using two metrics. The first one is the algorithm runtime. The lower the time it takes to run, the better the algorithm. Note that this does not include time taken for any file reading or post algorithm validation, since these aspects are not the focus of this research and can vary widely depending on the use case. The second metric we will be analyzing is the number of colors. This represents the quality of the coloring – lower colors mean the algorithm was more efficient. For both metrics, lower numbers are better.

4.1 Algorithm Runtime

Table 1 and Figure 1 describe and illustrate the differences in algorithm runtime between the 3 algorithms, where lower numbers are better.

Table 1: Algorithm Runtime in Seconds

Graph	NetworkX Greedy	Independent Set	Maximal Independent Set
as-Skitter	69.52	3.35	1.29
com-Youtube	21.81	1.27	0.57
com-LiveJournal	224.14	13.88	7.47
GAP-road	312.03	0.81	1.31

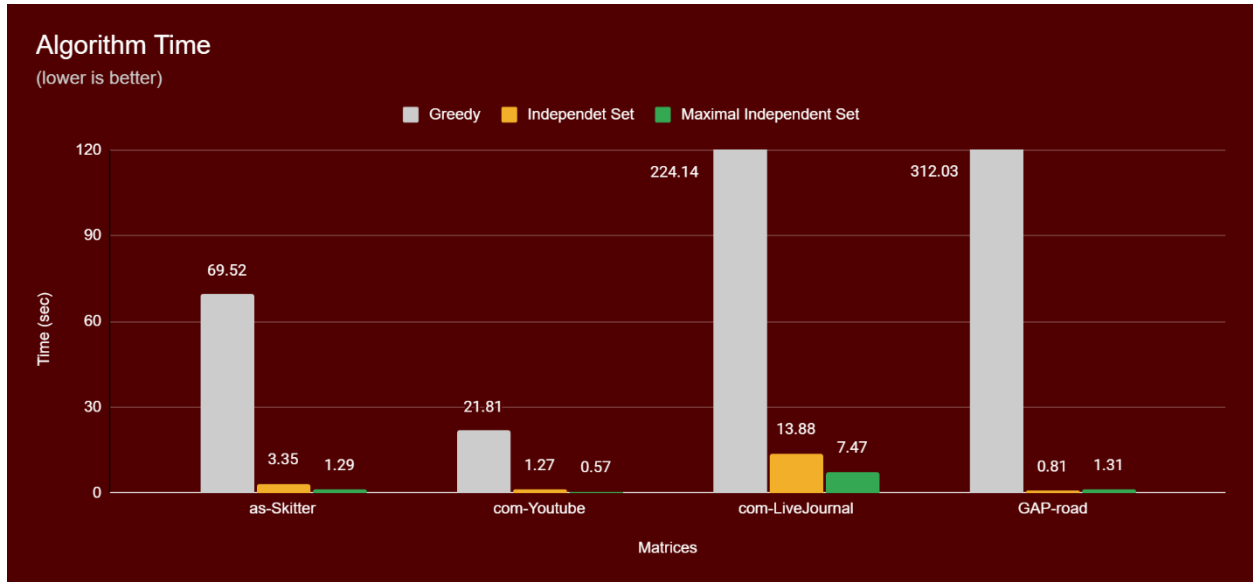


Figure 1: Algorithm Runtime Bar Graph

The algorithm runtimes from the results show that the Greedy NetworkX approach took the longest time, exceeding 300 seconds for a matrix that took the other algorithms less than a second to solve. This is a huge margin and highlights the power of GraphBLAS for running algorithms on large matrices. As graph size and edge count increase, parallel computation becomes not only useful, but essential to calculate a final result.

Between the maximal independent set algorithm and the independent set algorithm, the former beat the latter on 3 out of the 4 matrices. On GAP-road, the independent set algorithm beat the maximal independent set algorithm. This is an interesting situation, likely caused by the nature of the GAP-road graph. GAP-road is a very sparse graph, and finding an independent set is relatively straightforward. The overhead of the maximal independent set approach doesn't yield much benefit over the independent set algorithm – but the difference is around half a second. In practice, this difference is negligible and could vary with multiple iterations.

There were also some matrices that made it inconvenient to run the greedy algorithm and the Independent Set algorithm due to the extremely long runtime but were feasible with the

maximal independent set algorithm – however the inability to conclude data from the other two algorithms for those matrices led to those results being excluded from this paper.

4.2 Number of Colors

Table 2 and Figure 2 below describe and illustrate the differences in quality between the algorithms, measured by number of colors, where lower numbers are better.

Table 2: Number of Colors

Graph	NetworkX Greedy	Independent Set	Maximal Independent Set
as-Skitter	71	389	103
com-Youtube	32	259	60
com-LiveJournal	328	537	332
GAP-road	5	15	5

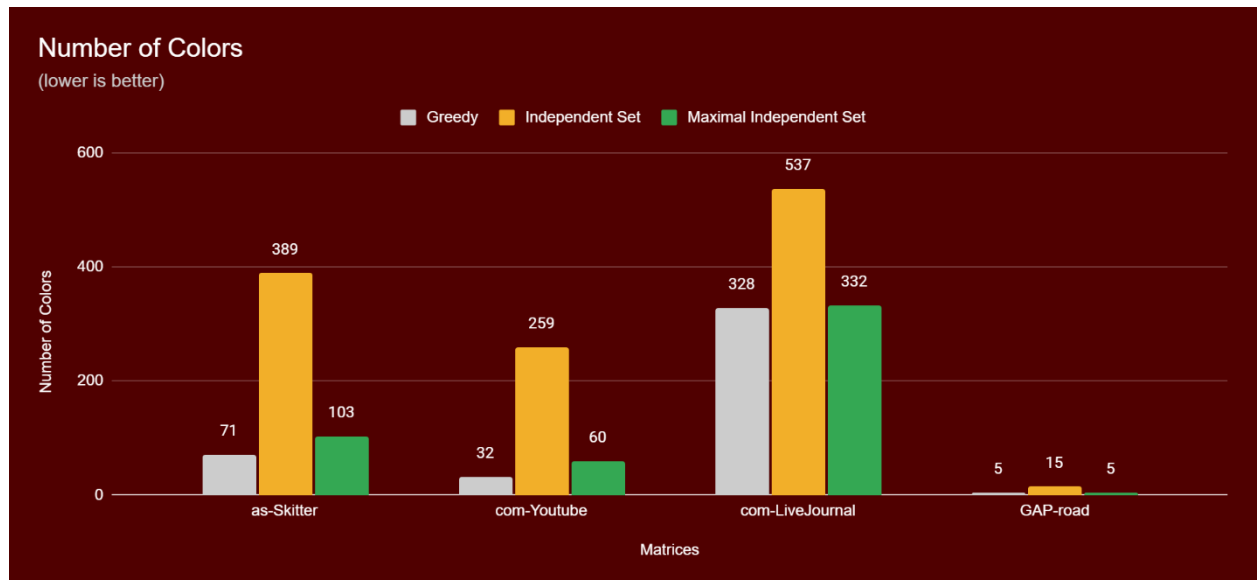


Figure 2: Number of Colors Bar Graph

From the graph, the independent set algorithm is the worst, producing significantly higher colors than the other two algorithms for each matrix. The NetworkX greedy algorithm uses the fewest colors, followed very closely by the maximal independent set algorithm.

Looking at both graphs, the maximal independent set algorithm is the clear winner for large graphs. While it may not provide the same efficiency as the greedy approach, it is clearly significantly faster and therefore much more practical in the real world. When faced with smaller matrices, the greedy algorithm can be used if time is not an issue and/or efficiency is paramount. The maximal independent set algorithm trumps the independent set algorithm in terms of runtime as well as coloring efficiency. This was expected because the maximal independent set algorithm is a direct improvement to the independent set algorithm. Despite this, the simplicity of the independent set algorithm can make it useful for situations where code simplicity is prioritized over raw performance.

5. CONCLUSION

GraphBLAS has proven to be a widely useful library to create and implement faster and more practical graph algorithms for large sparse matrices. The research in this paper has shown that it also has the potential to impact the field of graph coloring. We were able to find that the maximal independent set GraphBLAS coloring algorithm significantly beat the NetworkX greedy coloring algorithm in terms of time but was slightly overshadowed by the greedy algorithm in terms of efficiency. However, the maximal independent set algorithm is much more feasible for larger graphs and should be the preferred choice when dealing with large sparse matrices that would take far too long for a greedy algorithm to solve.

5.1 Challenges and Limitations

While the results from the maximal independent set algorithm are promising, there are limitations to this implementation. Firstly, the randomness factor of the algorithm can lead to variability in output, even with seeded values. Additionally, while the algorithm is efficient on large sparse graphs, dense graphs or graphs with certain structures can still result in poorer or negligible performance when compared to traditional greedy algorithms. Finally, the algorithm assumes a static input graph and does not support dynamic updates. In the real world, dynamic graphs are very common with the constant influx of data. Supporting incremental updates that can perform a partial re-coloring as opposed to coloring the entire graph again could be an extremely useful feature. Addressing these limitations could be an avenue for future researchers to explore.

5.2 Future Work

There are other parallel algorithms that exist for graph coloring. One that was investigated but did not make it into this paper was the Jones-Plassman algorithm. The algorithm changes the color assignment stage – instead of simply assigning the next available color for the independent set, the Jones-Plassman algorithm calls for each node to be assigned the lowest available color consistent with all of its neighbors. Integrating this into GraphBLAS would require two phases. The first would be to find an independent set with the existing maximal independent set logic. The second phase would be to assign each node in the set the minimum available color not used by any of its neighbors. The challenge is in the second phase, as GraphBLAS does not have a built-in way to solve this problem. One solution may involve unloading the matrix from GraphBLAS, processing externally in C, then loading the data back into the GraphBLAS matrix. An example of this implementation is provided in the paper in the related work section. There, the researchers define a “GxB_Scatter” operation. This is an experimental operation (denoted by the “x” in “GxB”) and involves pulling the code out of GraphBLAS and working on it in C. When doing this, the parallelism that GraphBLAS provides is essentially lost, as the code now goes back to a sequential workflow. However, there are still many algorithms in LAGraph that require this process of packing/unpacking a matrix in GraphBLAS. Furthermore, with the release of GraphBLAS v10 the load and unload functions have been introduced, offering a significant advancement over the previous pack and unpack functions and therefore potentially making this solution more viable.

An improvement to the user experience in LAGraph could also be an avenue for future work. Currently, users must manually ensure symmetry and remove self-edges. Before this

algorithm is uploaded to the official LA-Graph repository, there should be a user-friendly version that will automatically remove self-edges and ensure symmetry before running the algorithm.

REFERENCES

- [1] B. Brock, A. Buluç, T. Mattson, S. McMillan, and J. Moreira, *The GraphBLAS C API v2.0.0*, Nov. 2021. [Online]. Available: https://graphblas.org/docs/GraphBLAS_API_C_v2.0.0.pdf (accessed Apr. 3, 2025).
- [2] T. A. Davis, “Algorithm 1037: SuiteSparse:GraphBLAS: Parallel graph algorithms in the language of sparse linear algebra,” *ACM Trans. Math. Softw.*, vol. 49, no. 3, pp. 1–30, Sep. 2023, doi: 10.1145/3577195.
- [3] T. A. Davis, “Algorithm 1000: SuiteSparse:GraphBLAS: Graph algorithms in the language of sparse linear algebra,” *ACM Trans. Math. Softw.*, vol. 45, no. 4, pp. 1–25, Dec. 2019, doi: 10.1145/3322125.
- [4] GraphBLAS, “GitHub - GraphBLAS/LAGraph,” *GitHub*, Mar. 2025. [Online]. Available: <https://github.com/GraphBLAS/LAGraph> (accessed Apr. 3, 2025).
- [5] M. Osama, M. Truong, C. Yang, A. Buluç, and J. Owens, “Graph coloring on the GPU,” in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, Rio de Janeiro, Brazil, 2019, pp. 231–240, doi: 10.1109/IPDPSW.2019.00046.
- [6] Gunrock Project, “Gunrock: GPU graph analytics,” *GitHub*. [Online]. Available: <https://gunrock.github.io/gunrock> (accessed Apr. 3, 2025).
- [7] H. Mukesh, “LAGraph-coloring,” *GitHub*. [Online]. Available: <https://github.com/hmukesh5/LAGraph-coloring> (accessed Apr. 3, 2025).
- [8] NetworkX Developers, “networkx.algorithms.coloring.greedy_coloring,” *NetworkX Documentation*, [Online]. Available: https://networkx.org/documentation/stable/_modules/networkx/algorithms/coloring/greedy_coloring.html#greedy_color (accessed Apr. 3, 2025).
- [9] National Institute of Standards and Technology (NIST), “Matrix Market – text file formats,” *NIST*, [Online]. Available: <https://math.nist.gov/MatrixMarket/formats.html> (accessed Apr. 3, 2025).

APPENDIX: ALGORITHMS

Algorithm 1: Greedy Algorithm with NetworkX

Input: Matrix Market File
Output: Array of colors

- 1 $A = \text{mmread}(\text{filename})$
- 2 $G = \text{networkx.graph}(A)$
- 3 $C = \text{networkx.coloring.greedy_color}(G)$
- 4 **return** C

Algorithm 2: Independent Set Algorithm with GraphBLAS

Input: LAGraph_Graph object G with n nodes
Output: Array of colors C , Number of colors n_colors

- 1 **for** curr_color in $[1, n]$
- 2 initialize empty vectors max_weights and in_curr_subset of length n
initialize vector weight with random values of length n
- 3 $\text{GrB_mxv}(\text{output: max_weights, mask: local_color, accumulator: GrB_NULL, semiring: GrB_MAX_SECOND_SEMIRING_UNIT64, matrix: } G \rightarrow A, \text{ vector: weight descriptor: GrB_DESC_RSC})$
- 4 $\text{GrB_eWiseMult}(\text{output: in_curr_subset, mask: GrB_NULL, accumulator: GrB_NULL, operator: GrB_GT_UNIT64, vector: weight, vector: max_weights, descriptor: GrB_NULL})$
- 5 $\text{GrB_select}(\text{output: in_curr_subset, mask: GrB_NULL, accumulator: GrB_NULL, operator: GrB_VALUEEQ_BOOL, vector: in_curr_subset, scalar: true, descriptor: GrB_NULL})$
- 6 **if** in_curr_subset is empty:
- 7 **break**

Algorithm 2: Independent Set Algorithm with GraphBLAS, continued

```
8  GrB_assign(  
    output: local_color,  
    mask: in_curr_subset,  
    accumulator: GrB_NULL,  
    value: curr_color,  
    row_indices: GrB_ALL,  
    num_indices: n,  
    descriptor: GrB_DESC_S )  
9  GrB_assign(  
    output: weight,  
    mask: in_curr_subset,  
    accumulator: GrB_NULL,  
    value: 0,  
    row_indices: GrB_ALL,  
    num_indices: n,  
    descriptor: GrB_DESC_S )
```

Algorithm 3: Maximal Independent Set Algorithm with GraphBLAS

```
    Input: LAGraph_Graph object G with n nodes  
    Output: Array of colors C, Number of colors  $n\_colors$   
1  initialize local_color, mis  
2  for curr_color in [1, n]  
3  run LAGraph_Maximal_Independent_Set( G->A, mis )  
4  if mis is empty, break  
5  GrB_assign(  
    output: local_color,  
    mask: mis,  
    accumulator: GrB_NULL,  
    value: curr_color,  
    row_indices: GrB_ALL,  
    num indices: n,  
    descriptor: GrB_DESC_S )  
6  C = local_color  
7   $n\_colors = curr\_color - 1$ 
```
